

Full Name : Ghrib Ala Eddine
Class : Second Year of Bachelor's in Informatics
Group : 5(A)
Module : OS

TP2 :Prise en main Linux et Observation des Processus

Part 1 : Discovering The terminal

1 - Open a terminal (CTRL+ALT+T)

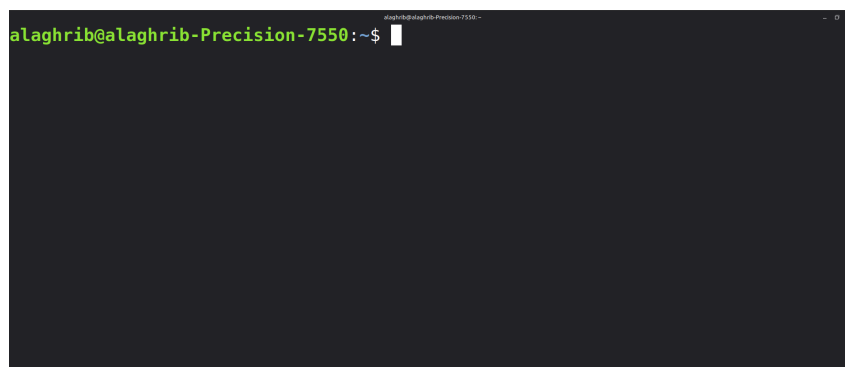


FIGURE 1 – Part 1-1 Screenshot

2 - Where Am I ?

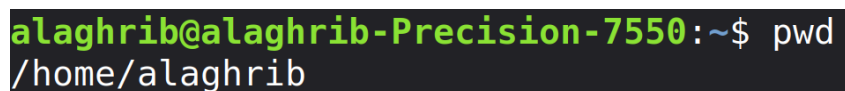


FIGURE 2 – Part 1-2 Screenshot

3 - List the files

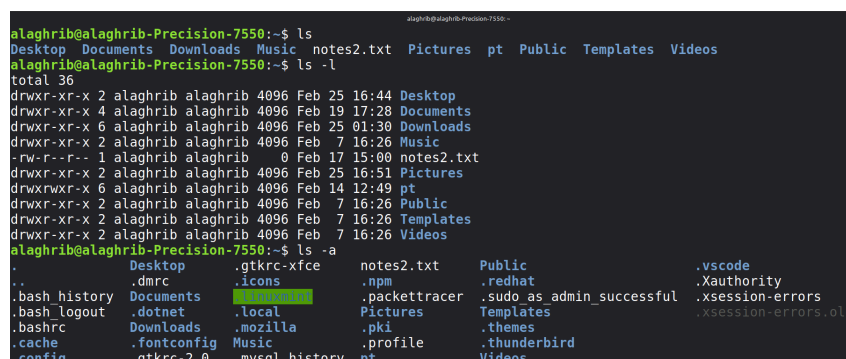
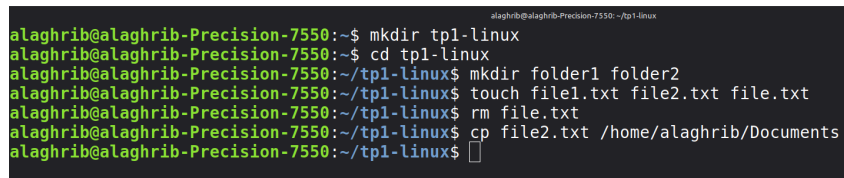


FIGURE 3 – Part 1-3 Screenshot

- 1 - What is the difference between **ls** and **ls -a** ?
ls : Shows the visible files.
ls -a : Shows all files, including hidden files.
(The hidden files start with .)
- 2 - What does the first column of **ls -l** mean ?
[(**drwxr-xr-x**),(**-rw-r-r-**),...] mean :
First character : File type
- = regular file
d = directory
The rest : Permissions for (**Owner,Group,Others**)
r = read
w = write
x = execute

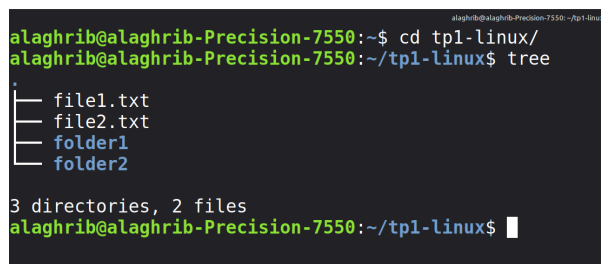
Part 2 : File navigation and management



```
alaghrib@alaghrib-Precision-7550: ~/tp1-linux
alaghrib@alaghrib-Precision-7550:~$ mkdir tp1-linux
alaghrib@alaghrib-Precision-7550:~$ cd tp1-linux
alaghrib@alaghrib-Precision-7550:~/tp1-linux$ mkdir folder1 folder2
alaghrib@alaghrib-Precision-7550:~/tp1-linux$ touch file1.txt file2.txt file.txt
alaghrib@alaghrib-Precision-7550:~/tp1-linux$ rm file.txt
alaghrib@alaghrib-Precision-7550:~/tp1-linux$ cp file2.txt /home/alaghrib/Documents
alaghrib@alaghrib-Precision-7550:~/tp1-linux$
```

FIGURE 4 – Part 2 Screenshot

- 1 - Create a folder named **tp1-linux** .
" mkdir tp1-linux "
- 2 - Change directory to **/tp1-linux** .
"cd tp1-linux "
- 3 - Create 2 Folders inside it named **folder1** , **folder2** .
"mkdir folder1 folder2 "
- 4 - Add 3 text files named **file1.txt** , **file2.txt** , **file.txt** .
"touch file1.txt file2.txt file.txt"
- 5 - Remove a file **file.txt** .
"rm file.txt"
- 6 - Copy a file **file2.txt** to **/home/alaghrib/Documents** .
"cp file2.txt /home/alaghrib/Documents"



```
alaghrib@alaghrib-Precision-7550:~$ cd tp1-linux/
alaghrib@alaghrib-Precision-7550:~/tp1-linux$ tree
.
├── file1.txt
├── file2.txt
├── folder1
└── folder2

3 directories, 2 files
alaghrib@alaghrib-Precision-7550:~/tp1-linux$
```

FIGURE 5 – tree

Part 3 : Background Programs

1 - Finding the process "sleep"

```
alaghr@alaghr-Precision-7550:~$ ps aux
:1-rb_allocator]
root      8961  0.0  0.0      0   0 ?        I   19:08   0:00 [kworker/0:1
-:events]
root      9016  0.0  0.0      0   0 ?        I   19:08   0:00 [kworker/1:2
-:events]
root      9018  0.0  0.0      0   0 ?        I   19:08   0:00 [kworker/2:1
-mm_percpu_wq]
root      9023  0.0  0.0      0   0 ?        I   19:09   0:00 [kworker/11:
1-1915-unorderd]
root      9047  0.0  0.0      0   0 ?        I   19:10   0:00 [kworker/8:0
-:events]
root      9049  0.0  0.0      0   0 ?        I   19:10   0:00 [kworker/7:0
]
root      9071  0.0  0.0      0   0 ?        I   19:11   0:00 [kworker/3:0
]
alaghr@alaghr-Precision-7550:~$ ps aux
alaghr@alaghr-Precision-7550:~$
```

FIGURE 6 – The process "sleep"

2 - The process PID : 9072

3 - The 'STAT' column mean : process state

- S = sleeping
- R = running
- T = stopped
- Z = zombie
- += foreground

Part 4 : CPU monitoring

```
alaghr@alaghr-Precision-7550:~$ top
top - 07:10:44
  PID TID          PPID  %CPU   %MEM    VSZ   RSS  Cmd
  16098 alaghr  20  97.7  1.0  3418.92  1000  /usr/lib/firefox/firefox-bin -contentproc parentBuildID 4051111111
  20130 alaghr  20  0.0  0.0  0.0  0.0  /usr/lib/firefox/firefox-bin -contentproc parentBuildID 4051111111
  7501 alaghr  20  0.0  0.0  0.0  0.0  /usr/lib/firefox/firefox-bin -contentproc parentBuildID 4051111111
  7832 alaghr  20  0.0  0.0  0.0  0.0  /usr/lib/firefox/firefox-bin -contentproc parentBuildID 4051111111
  7836 alaghr  20  0.0  0.0  0.0  0.0  /usr/lib/firefox/firefox-bin -contentproc parentBuildID 4051111111
  7933 alaghr  20  0.0  0.0  0.0  0.0  /usr/lib/firefox/firefox-bin -contentproc parentBuildID 4051111111
  1      1      0  0.0  0.0  0.0  0.0  /bin/init splash
  303 root    19  0.0  0.0  0.0  0.0  /usr/lib/systemd/systemd-journald
  412 root    0  0.0  0.0  0.0  0.0  /usr/lib/systemd/systemd-udev
  692 systemd-re 20  0.0  0.0  0.0  0.0  /usr/lib/systemd/systemd-resolved
  702 systemd-l 20  0.0  0.0  0.0  0.0  /usr/lib/systemd/systemd-timesyncd
  748 systemd-l 20  0.0  0.0  0.0  0.0  /usr/lib/systemd/systemd-timesyncd
  752 root    0  0.0  0.0  0.0  0.0  /usr/libexec/accounts-daemon
  755 root    0  0.0  0.0  0.0  0.0  /usr/libexec/evemu-demon running [alaghr-Precision-7550.local]
  757 root    0  0.0  0.0  0.0  0.0  /usr/libexec/bluetooth/bluetoothd
  758 root    0  0.0  0.0  0.0  0.0  /usr/libexec/cron -f
  759 root    0  0.0  0.0  0.0  0.0  /usr/libexec/daemon --system --address-systemd: --nofork --nopidfile --systemd
  764 root    0  0.0  0.0  0.0  0.0  /usr/libexec/daemon
  766 root    0  0.0  0.0  0.0  0.0  /usr/libexec/daemon
  767 root    0  0.0  0.0  0.0  0.0  /usr/libexec/daemon
  771 polkitd  20  0.0  0.0  0.0  0.0  /usr/lib/polkit-1/polkitd --no-debug
  773 root    0  0.0  0.0  0.0  0.0  /usr/libexec/power-profiles-daemon
  781 root    0  0.0  0.0  0.0  0.0  /usr/libexec/switcheroo-control
```

FIGURE 7 – The most CPU

- 1 - Process consuming the most CPU : (User : alaghr , Command : firefox-bin , CPU = 97,7) .
- 2 - Its PID = 16098 .
- 3 - PRESS 'F10' to exit 'TOP'.

Part 5 : Process Tree

1 - The root process : **systemd** with PID = 1.

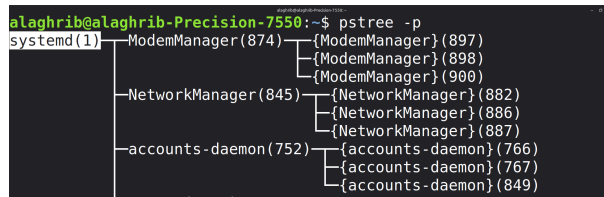


FIGURE 8 – The root process

2 - My Shell is : **bash(22865)**

located under : **systemd(1)/gnome-terminal(22854)/bash(22865)** .

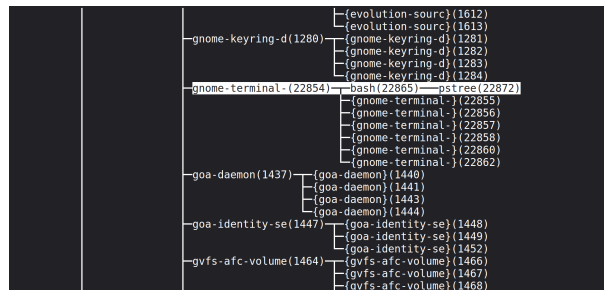


FIGURE 9 – The shell location

3 - The child process : **pstree(22872)** is a child of **bash(22865)**

Part 6 : Starting and stopping a program

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
25415	alaghrib	20	0	8292	2064	1944	R	99.7	0.0	0:00.61
6861	alaghrib	20	0	13.1g	699532	285188	S	4.3	4.4	57:39.74
1488	alaghrib	20	0	4727692	234444	121592	S	7.7	1.4	22:45.30
1027	root	20	0	819320	122376	76668	S	1.7	0.8	9:12.60
21914	alaghrib	20	0	4442700	521496	134616	S	1.0	3.3	3:27.30
105	root	-51	0	0	0	0	S	0.7	0.0	3:05.75
11885	root	20	0	0	0	0	I	0.7	0.0	0:04.85
22261	root	0	20	0	0	0	I	0.7	0.0	0:09.19
25361	alaghrib	20	0	691448	44944	34716	S	0.7	0.3	0:01.02
81	root	20	0	0	0	0	S	0.3	0.0	2:12.09
412	root	20	0	30104	8252	4988	S	0.3	0.1	3:35.59
23515	root	20	0	0	0	0	I	0.3	0.0	0:05.64
25097	root	20	0	0	0	0	I	0.3	0.0	0:00.09
1	root	20	0	22720	13992	9584	S	0.0	0.1	0:00.21
2	root	20	0	0	0	0	S	0.0	0.0	0:00.05
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00
4	root	0	20	0	0	0	I	0.0	0.0	0:00.00
5	root	0	20	0	0	0	I	0.0	0.0	0:00.00
6	root	0	20	0	0	0	I	0.0	0.0	0:00.00
7	root	0	20	0	0	0	I	0.0	0.0	0:00.00
8	root	0	20	0	0	0	I	0.0	0.0	0:00.00
10	root	0	20	0	0	0	I	0.0	0.0	0:00.00

FIGURE 10 – Part 6

1 - The Difference between kill and kill -9 :

kill sends a polite request (called SIGTERM (15)), process can ignore it .

Kill -9 sends a forced request (called SIGKILL (9)), process cannot be ignored.

2 - "**YES**" consume so much CPU because it runs an infinite loop printing "yes" non-stop with no pause, it uses 100 % CPU.

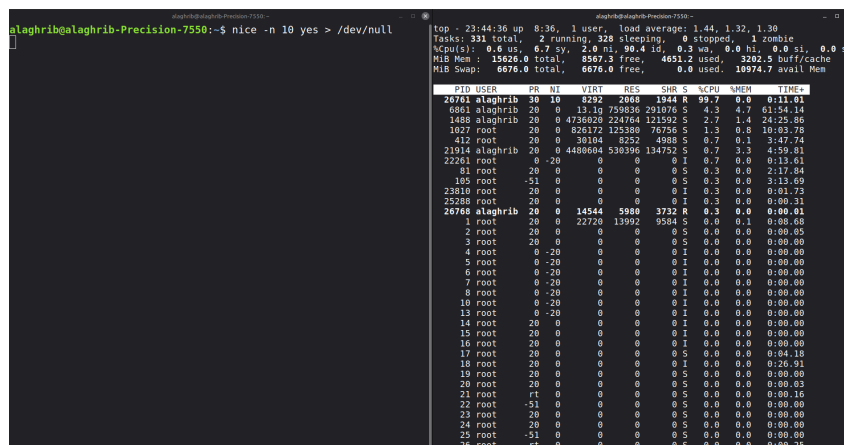
Part 7 : Backgrounds Jobs

```
alaghrib@alaghrib-Precision-7550:~$ sleep 200 &
[1] 26069
alaghrib@alaghrib-Precision-7550:~$ jobs
[1]+  Running                  sleep 200 &
alaghrib@alaghrib-Precision-7550:~$ fg
sleep 200
^Z
[1]+  Stopped                  sleep 200
alaghrib@alaghrib-Precision-7550:~$
```

FIGURE 11 – Part 7

- **sleep 200** and runs the process in the background with PID = 26069 . **jobs** shows it as Running. **fg** brings it to the foreground. **Ctrl+Z** suspends it so status changes to Stopped.

Part 8 : Priority and CPU



PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
26761	alaghrib	30	10	8292	2068	1944	R	99.7	0.0	0:11.01
1861	alaghrib	20	0	13.1g	759336	231076	S	4.2	4.7	0:1:54.14
1488	alaghrib	20	0	4736020	224764	121592	S	2.7	1.4	24:25.86
1027	root	20	0	826172	125388	76756	S	1.2	0.8	10:03.78
412	root	20	0	30104	8252	4988	S	0.7	0.1	3:47.74
21914	alaghrib	20	0	4480604	538396	134752	S	0.7	3.3	4:59.81
22281	root	0	-20	0	0	0	S	0.7	0.0	0:13.61
81	root	20	0	0	0	0	S	0.3	0.0	2:17.84
105	root	-51	0	0	0	0	S	0.3	0.0	3:13.69
23310	root	20	0	0	0	0	S	0.3	0.0	0:21.73
25288	root	20	0	0	0	0	S	0.3	0.0	0:00.31
26768	alaghrib	20	0	14544	5988	3732	R	0.3	0.0	0:00.01
1	root	20	0	22720	13992	3584	S	0.0	0.1	0:00.08
2	root	20	0	0	0	0	S	0.0	0.0	0:00.05
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00
4	root	0	-20	0	0	0	S	0.0	0.0	0:00.00
5	root	0	-20	0	0	0	S	0.0	0.0	0:00.00
6	root	0	-20	0	0	0	S	0.0	0.0	0:00.00
7	root	0	-20	0	0	0	S	0.0	0.0	0:00.00
8	root	0	-20	0	0	0	S	0.0	0.0	0:00.00
10	root	0	-20	0	0	0	S	0.0	0.0	0:00.00
13	root	0	-20	0	0	0	S	0.0	0.0	0:00.00
14	root	20	0	0	0	0	S	0.0	0.0	0:00.00
15	root	20	0	0	0	0	S	0.0	0.0	0:00.00
16	root	20	0	0	0	0	S	0.0	0.0	0:00.00
17	root	20	0	0	0	0	S	0.0	0.0	0:04.18
18	root	20	0	0	0	0	S	0.0	0.0	0:26.91
19	root	20	0	0	0	0	S	0.0	0.0	0:00.00
20	root	20	0	0	0	0	S	0.0	0.0	0:00.03
21	root	rt	0	0	0	0	S	0.0	0.0	0:00.16
22	root	-51	0	0	0	0	S	0.0	0.0	0:00.00
23	root	20	0	0	0	0	S	0.0	0.0	0:00.00
24	root	20	0	0	0	0	S	0.0	0.0	0:00.00
25	root	-51	0	0	0	0	S	0.0	0.0	0:00.00
26	root	rt	0	0	0	0	S	0.0	0.0	0:00.25

FIGURE 12 – Part 8

1 - The priority :

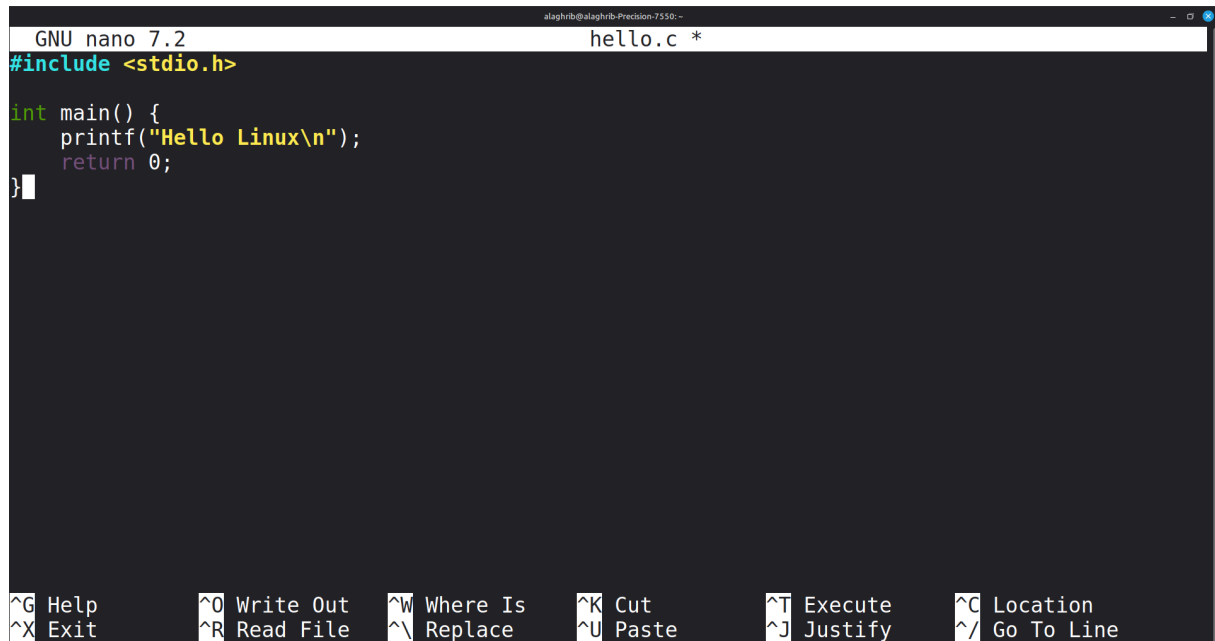
The process "**yes**" was launched with '**nice -n 10**', so its nice value is NI = 10 and its internal priority is PR = 30. This means it has a lower priority than a normal process (NI = 0, PR = 20).

2 - Comparison with a normal process : A normal process runs with NI = 0 and PR = 20. Our yes process runs with NI = 10 and PR = 30. This means if both processes compete for the CPU at the same time, the normal process will get CPU time first, and yes will wait.

3 - What does nice do ?

nice sets the scheduling priority of a process. Range : -20 (highest) to +19 (lowest). nice -n 10 means the process has lower priority and gets less CPU time when competing with others.

Part 9 : Simple Compilation

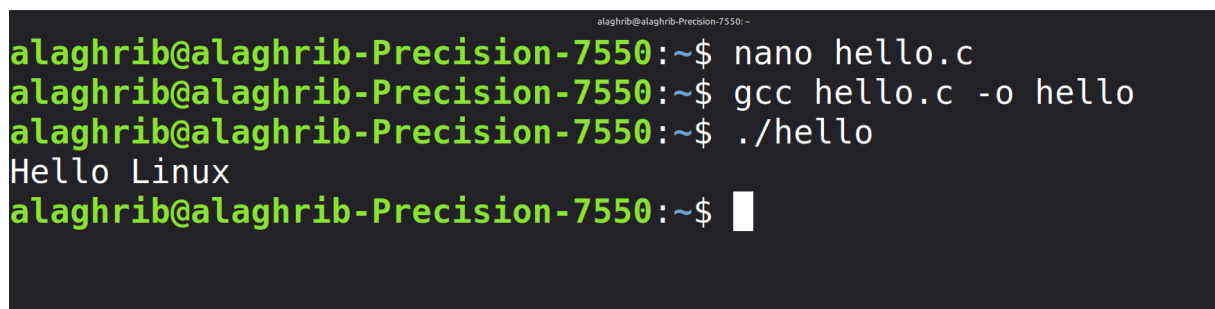


```
GNU nano 7.2 hello.c *
#include <stdio.h>

int main() {
    printf("Hello Linux\n");
    return 0;
}
```

The screenshot shows the nano 7.2 text editor interface. The title bar indicates the file is 'hello.c *'. The code is written in a dark-themed editor with syntax highlighting: '#include' is blue, '<stdio.h>' is orange, 'int' is green, 'main()' is green, 'printf' is green, 'Hello Linux\n' is orange, 'return' is green, and '0;' is green. The bottom status bar shows various keyboard shortcuts for nano editor functions.

FIGURE 13 – The code



```
alaghrib@alaghrib-Precision-7550: ~$ nano hello.c
alaghrib@alaghrib-Precision-7550: ~$ gcc hello.c -o hello
alaghrib@alaghrib-Precision-7550: ~$ ./hello
Hello Linux
alaghrib@alaghrib-Precision-7550: ~$
```

The screenshot shows a terminal window with the following commands and output: 'nano hello.c' (creates the file), 'gcc hello.c -o hello' (compiles the file into an executable named 'hello'), and './hello' (runs the executable, outputting 'Hello Linux'). The prompt is 'alaghrib@alaghrib-Precision-7550: ~\$'.

FIGURE 14 – The creation and Compile